Project Report

Comparison between PostgreSQL and Neo4j for AI Research Paper Retrieval and Knowledge Graph Analysis

Data Management 2025/2026

Students	Hamza Abdul Kader, matricola 2230150
	Leonardo Marasca, matricola 2217140

Project type	NoSQL / DBMS comparison
Dataset	OpenAlex AI research paper subset
Systems compared	PostgreSQL 16 and Neo4j 5

This report is the polished PDF version of the living project documentation.

Contents

1 Executive Summary

This project compares a relational database management system and a graph database using the same dataset of AI-related research papers collected from OpenAlex. The relational implementation uses PostgreSQL, while the graph implementation uses Neo4j.

The main objective is not only to check which system is faster, but also to understand how each system represents and queries relationship-heavy scholarly data. Scientific papers naturally form a graph: authors write papers, papers cite other papers, papers have topics, and authors become connected through shared publications, topics, and citations.

The work completed so far includes:

- collecting and cleaning an OpenAlex subset focused on AI, Machine Learning, Large Language Models, and Retrieval-Augmented Generation;
- modeling the same data in PostgreSQL and Neo4j;
- loading both databases with the same CSV files;
- validating that both databases contain matching entity and relationship counts;
- writing equivalent SQL and Cypher queries;
- benchmarking 11 query pairs;
- generating charts and tables for presentation.

The current benchmark shows that PostgreSQL is faster on most queries for this small dataset, especially ranking and aggregation queries. Neo4j is faster on the author citation network query, where the traversal follows a natural author-to-paper-to-paper-to-author graph pattern. The broader conclusion is that PostgreSQL is very efficient for tabular analysis, while Neo4j is often clearer and more direct for graph-shaped questions.

2 Project Goal and Research Questions

The goal of the project is to compare PostgreSQL and Neo4j on the same AI research paper dataset. The comparison focuses on data modeling, query complexity, expressiveness, execution time, and suitability for relationship-heavy analysis.

The main research questions are:

1. How can the same scholarly dataset be modeled in a relational database and in a graph database?
2. Which system makes common analytical queries easier to express?
3. Which system performs better on the selected dataset and query workload?
4. Are graph-shaped queries more naturally represented in Neo4j than in PostgreSQL?
5. What are the practical trade-offs between relational joins and graph traversals?

3 Dataset

3.1 Source

The dataset source is OpenAlex, an open catalog of scholarly works, authors, institutions, topics, citations, and related academic metadata.

- Main website: `https://openalex.org/`
- Dataset documentation: OpenAlex About the data

OpenAlex is appropriate for this project because it already contains the types of entities and relationships that are useful for comparing relational and graph databases:

- papers and their metadata;
- authorship relationships;
- topics and concepts;
- citation links between papers;
- publication years, titles, abstracts, and citation counts.

3.2 Scope of the Collected Subset

The full OpenAlex dataset is too large for a student project, so we collect a focused subset through the OpenAlex API. The selected subset is related to:

- retrieval augmented generation;
- large language models;
- artificial intelligence;
- machine learning.

The current sample was collected with:

```
python src/collect_openalex.py --per-term 75 --per-page 75
```

Table 1: Current OpenAlex dataset sample

Entity	**Count**
Papers	299
Authors	1966
Topics	1104
Paper-author relationships	2077
Paper-topic relationships	4288
Citation relationships	392

Generated data files are stored locally in `data/raw/` and `data/processed/`. They are ignored by git because they can be regenerated from the OpenAlex API.

4 Data Collection and Cleaning Pipeline

The collection script is `src/collect_openalex.py`. It uses only the Python standard library so the project can run without dependency installation.

The pipeline performs the following operations:

1. Calls the OpenAlex Works API using AI-related search terms.
2. Uses cursor pagination to retrieve multiple pages of results.
3. Saves raw API objects as JSONL.
4. Reconstructs abstracts from OpenAlex abstract inverted indexes when available.
5. Extracts paper metadata such as title, year, DOI, OpenAlex ID, citation count, and matched search terms.
6. Extracts authors and paper-author relationships.
7. Extracts topics and paper-topic relationships.
8. Extracts citations where both the citing and cited papers are inside the selected subset.
9. Removes duplicate paper-author pairs before database loading.
10. Writes normalized CSV files for both PostgreSQL and Neo4j.

Table 2: Generated CSV files

File	**Content**
`papers.csv`	One row per selected OpenAlex work.
`authors.csv`	One row per author.
`topics.csv`	One row per topic or concept.
`paper_authors.csv`	Many-to-many relationship between papers and authors.
`paper_topics.csv`	Many-to-many relationship between papers and topics.
`citations.csv`	Citation links where both papers are inside the selected subset.

4.1 Data Quality Issue

During PostgreSQL loading, the project found that OpenAlex can contain duplicate author entries for the same paper. This produced a duplicate primary key error in the `paper_authors` table.

The fix was to de-duplicate paper-author relationships by `(paper_id, author_id)` during normalization. This is a useful project detail because it shows a realistic data cleaning problem and how it was solved.

5 Repository Structure

```
src/
  collect_openalex.py
  benchmark_queries.py
```

```
  generate_benchmark_charts.py

database/postgres/
  schema.sql
  load.sql
  queries.sql

database/neo4j/
  constraints.cypher
  import.cypher
  queries.cypher

benchmarks/
  queries.json
  results/benchmark_results.csv

docs/
  project_proposal.pdf
  project_roadmap.pdf
  project_report.md
  project_report.tex
  project_report.pdf
  benchmark_results.md
  figures/
```

6 PostgreSQL Relational Model

PostgreSQL uses a normalized relational schema. Entities are represented as tables, and many-to-many relationships are represented as bridge tables.

Table 3: PostgreSQL tables

Table	**Purpose**
`papers`	Stores paper metadata such as title, year, DOI, abstract, topic, citation count, and OpenAlex URL.
`authors`	Stores author IDs and display names.
`topics`	Stores topic IDs and names.
`paper_authors`	Bridge table connecting papers and authors.
`paper_topics`	Bridge table connecting papers and topics.
`citations`	Stores directed paper-to-paper citation links.

This model is strong for:

- aggregations;
- filtering;
- sorting;
- structured joins;

- enforcing primary and foreign keys.

However, graph-shaped questions can become verbose because they require multiple joins or recursive queries.

7 Neo4j Graph Model

Neo4j uses nodes and relationships. This representation is closer to the natural structure of scholarly data.

Table 4: Neo4j graph model

Graph element	**Meaning**
`(:Paper)`	Research paper node.
`(:Author)`	Author node.
`(:Topic)`	Topic node.
Author → Paper	Authorship relationship, implemented as `AUTHORED`.
Paper → Topic	Topic assignment relationship, implemented as `HAS_TOPIC`.
Paper → Paper	Directed citation relationship, implemented as `CITES`.

Example graph patterns:

```
(Author)-[:AUTHORED]->(Paper)
(Paper)-[:HAS_TOPIC]->(Topic)
(Paper)-[:CITES]->(Paper)
```

This model is strong for:

- citation paths;
- author collaboration;
- shared topics;
- author citation networks;
- relationship traversal.

8 Database Setup and Loading

The project uses Docker Compose to run both systems locally.

Table 5: Docker services

Service	**Image**	**Access**
PostgreSQL	`postgres:16`	`localhost:5432`
Neo4j	`neo4j:5`	`http://localhost:7474`

Start the databases:

```
docker compose up -d
```

Load PostgreSQL:

```
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/schema.sql
docker exec openalex-postgres psql -U openalex -d openalex_ai -f /sql/load.sql
```

Load Neo4j:

```
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
  -f /cypher/constraints.cypher
docker exec openalex-neo4j cypher-shell -u neo4j -p openalex123 \
  -f /cypher/import.cypher
```

9 Validation

After loading both databases, the counts were compared to ensure that the same dataset was present in both systems.

Table 6: Validation of loaded data

Data element	**PostgreSQL**	**Neo4j**
Papers	299	299
Authors	1966	1966
Topics	1104	1104
Paper-author / AUTHORED	2077	2077
Paper-topic / HAS_TOPIC	4288	4288
Citations / CITES	392	392

The counts match exactly, confirming that the benchmark compares the same logical data in both systems.

10 Query Workload

The benchmark contains 11 paired SQL/Cypher queries. They are stored in `benchmarks/queries.json`.

Table 7: Benchmark query workload

ID	**Query**	**Purpose**
Q1	Most cited papers	Rank papers by citation count.
Q2	Authors with the most papers	Count selected papers per author.
Q3	Most frequent topics	Count selected papers per topic.
Q4	Author collaboration pairs	Find authors who worked together most often.
Q5	Citation links inside the subset	Return directed citation relationships.
Q6	RAG-related papers	Search title and abstract for RAG-related terms.
Q7	Two-hop citation paths	Find papers connected by a two-step citation path.
Q8	Authors connected through shared topics	Find author pairs with many shared topics.

ID	Query	Purpose
Q9	Papers sharing cited references	Find papers that cite the same references.
Q10	Citation paths up to three hops	Find paper pairs connected by citation paths of length two or three.
Q11	Author citation network	Find authors connected when one author's papers cite another author's papers.

11 Benchmark Methodology

The benchmark is run with:

```
python src/benchmark_queries.py --runs 5 --warmups 1
```

The methodology is:

- one warmup run per query/system pair;
- five measured runs per query/system pair;
- PostgreSQL timing uses `EXPLAIN (ANALYZE, FORMAT JSON)`;
- Neo4j timing uses `PROFILE` through `cypher-shell --format verbose`;
- raw results are saved in `benchmarks/results/benchmark_results.csv`;
- Markdown results are saved in `docs/benchmark_results.md`.

This benchmark should be interpreted as a local project benchmark, not as a universal ranking of PostgreSQL and Neo4j. Performance depends on data size, indexing, cache state, hardware, and query shape.

12 Benchmark Results

Table 8: Average benchmark execution time

Query	PostgreSQL avg ms	Neo4j avg ms	Faster system
Most cited papers	0.058	4.200	PostgreSQL
Authors with the most papers	2.422	8.600	PostgreSQL
Most frequent topics	4.762	15.200	PostgreSQL
Author collaboration pairs	44.358	65.000	PostgreSQL
Citation links inside the subset	0.183	4.400	PostgreSQL
RAG-related papers	3.515	8.600	PostgreSQL
Two-hop citation paths	1.577	8.400	PostgreSQL
Authors connected through shared topics	309.447	586.600	PostgreSQL
Papers sharing cited references	2.171	7.800	PostgreSQL
Citation paths up to three hops	5.104	12.400	PostgreSQL
Author citation network	48.214	37.400	Neo4j

13 Benchmark Graphs

Figure 1: Average query time comparison

Query	PostgreSQL	Neo4j
Most cited papers	0.058 ms	4.2 ms
Top authors	2.4 ms	8.6 ms
Top topics	4.8 ms	15.2 ms
Author collaboration	44.4 ms	65.0 ms
Citation links	0.183 ms	4.4 ms
RAG-related papers	3.5 ms	8.6 ms
Two-hop citations	1.6 ms	8.4 ms
Shared topics	309.4 ms	586.6 ms
Shared references	2.2 ms	7.8 ms
Paths 2–3 hops	5.1 ms	12.4 ms
Author citation network	48.2 ms	37.4 ms

Figure 2: Relative query time: Neo4j average divided by PostgreSQL average

Query	Relative time
Most cited papers	72.4x
Top authors	3.6x
Top topics	3.2x
Author collaboration	1.5x
Citation links	24.0x
RAG-related papers	2.4x
Two-hop citations	5.3x
Shared topics	1.9x
Shared references	3.6x
Paths 2–3 hops	2.4x
Author citation network	0.8x

Figure 1 shows absolute average execution time. Figure 2 shows how many times slower or faster Neo4j was relative to PostgreSQL. Values above 1x mean Neo4j was slower than PostgreSQL; values below 1x mean Neo4j was faster.

14 Interpretation

The benchmark has three important interpretations.

14.1 PostgreSQL is strong for tabular aggregation

Queries such as most cited papers, most frequent topics, and citation links are very efficient in PostgreSQL. These queries match the strengths of a relational DBMS: indexes, joins, grouping, and ordering.

14.2 Neo4j is clearer for graph-shaped questions

Neo4j's Cypher syntax is often easier to read when the question is naturally expressed as a path. For example, an author citation network can be expressed as:

```
(Author)-[:AUTHORED]->(Paper)-[:CITES]->(Paper)<-[:AUTHORED]-(Author)
```

In PostgreSQL, the same query requires several joins across `paper_authors`, `citations`, and `authors`.

14.3 The result depends on dataset size and query shape

The current dataset is intentionally small. Therefore, the results should not be interpreted as a general statement that PostgreSQL is always faster. With larger graphs, deeper traversals, more complex path search, or additional graph indexes/projections, the comparison could change.

15 Main Findings

- The same OpenAlex dataset was successfully represented in both PostgreSQL and Neo4j.
- PostgreSQL and Neo4j were loaded with matching entity and relationship counts.
- PostgreSQL was faster on 10 out of 11 benchmarked queries.
- Neo4j was faster on the author citation network query.
- Neo4j query syntax is often more natural for relationship traversal.
- PostgreSQL is highly efficient for relational aggregation and sorting.
- The most useful comparison is not only speed, but also modeling clarity and query expressiveness.

16 Limitations

- The dataset is a selected subset, not the full OpenAlex dataset.
- Benchmark results were produced on a local machine and depend on local hardware.
- Only 11 queries were benchmarked.
- The benchmark focuses on server-reported query time, not full application-level latency.
- The Neo4j model uses the basic graph database; advanced graph data science projections were not used.

17 Future Work

- Increase the dataset size and rerun the benchmark.
- Add deeper citation path queries.
- Add more RAG-specific retrieval queries.
- Add visual graph exploration examples in Neo4j Browser.
- Prepare the final presentation slides and live demo.

18 Conclusion

This project demonstrates a practical comparison between PostgreSQL and Neo4j using AI research paper data from OpenAlex. The relational model is efficient and structured, while the graph model is expressive and natural for relationship-heavy analysis.

The current results show PostgreSQL as faster for most queries on the small benchmark dataset, but Neo4j provides clearer graph traversal semantics and outperforms PostgreSQL on the author citation network query. This supports a balanced conclusion: PostgreSQL is a strong choice for structured analytical workloads, while Neo4j becomes especially valuable when the main questions are about relationships, paths, and networks.